

**Project Title: Automated Color-Sorting
Conveyor Belt**

Name: Muhammad Anas

Date: June 2026

Abstract

This report presents the design and development of an automated color sorting conveyor belt system. The system uses an Arduino Uno microcontroller, a TCS3200 color sensor, a DC motor-driven conveyor belt, and a servo-controlled sorting flap to detect and separate objects based on their color (red, green, blue and yellow). The programming logic was written in C++ over Arduino IDE, reads RGB values from the sensor. It determines the object's color, and activates the servo mechanism at the appropriate time to divert the object into the correct collection bin. Experimental testing showed accurate sorting with a success rate of 93% under controlled lighting conditions. The project demonstrates fundamental concepts in sensor integration, actuator control, real-time decision-making, and hardware-software interfacing. This work serves as a practical example of industrial automation principles applied at a prototype scale.

List of Figures

Figure 1: Circuit diagram (made with proteus software).....	7
---	---

List of Tables

Table 1: Project cost.....	6
Table 2: Pin map table.....	7
Table 3: Test results.....	8
Table 4: User defined functions for code.....	8
Table 5: Challenges faced during project and their implemented solutions.....	13

Table of Contents

Color Sorting Conveyor Belt.....	6
1. Introduction.....	6
2. Objectives.....	6
3. Tools and Components Required.....	6
3.1 Components List and Cost.....	6
3.2 Tools Required.....	6
4. Methodology & Circuit Diagram.....	7
4.1 Circuit.....	7
4.2 Pin mapping.....	7
4.3 Methodology.....	7
5. Fabrication & Testing Procedure.....	8
5.1 Fabrication process.....	8
5.2 Testing process.....	8
6. Program Code.....	8
7. Challenges and their solutions.....	13
Conclusions.....	14
References.....	15

Color Sorting Conveyor Belt

1. Introduction

The Color Sorting Conveyor Belt system is an automated mechanism that detects the color of objects on a moving conveyor belt and sorts them into different bins [1]. Color sensing matters in industry as its one of the methods of sorting objects [2]. Industrial sorting relies on automation, mechanical or manual processes to sort objects using weight, size, shape, color or destination. It can be used for manufacturing, recycling, food processing [3] etc. For instance, automatic sorting of fruits based on colors has been implemented. All in all, manual sorting is prone to many errors. And it is slow as a matter of fact. Automation solves this by making sure minimal manual work during operation, so less sorting errors happen.

Regardless, our project is related to one of those automation processes, it involves sensing color to sort objects into their specific bins. This project integrates hardware (sensors [4, 5], servo motors [6], conveyor belt [7]) with software (microcontroller programming [8]) to demonstrate real-world industrial automation principles on a small scale. The objective of this project is to implement a low-cost, Arduino-based color sorting conveyor belt solution.

2. Objectives

- Detect colors (Red, Green, Blue and Yellow) using a color sensor.
- Control conveyor belt movement and divert objects using servo mechanism.
- Provide a low cost solution
- Implement logic in code to sort objects accurately.

3. Tools and Components Required

3.1 Components List and Cost

Table 1: Project cost

Sr no.	Component	Specification	Quantity	Cost
1.	Microcontroller	Arduino Uno R3	1	1300
2.	Color Sensor	TCS3200	1	400
3.	DC motor		1	600
4.	Motor Driver	L298N	1	400
5.	Micro Servo	SG90	2	700
6.	Conveyor Belt		1	100
7.	Base	Wooden		1000
8.	Power Supply	Li-ion cells	4	1200
9.	BMS	4S BMS	1	350
10.	Jumper wires and Breadboard	As required	As required	300
11.	Miscellaneous	As required	As required	1000
Total Cost:				7350

3.2 Tools Required

- Soldering Iron, Soldering paste and wire
- Hot Glue Gun

- Cutting tools

4. Methodology & Circuit Diagram

4.1 Circuit

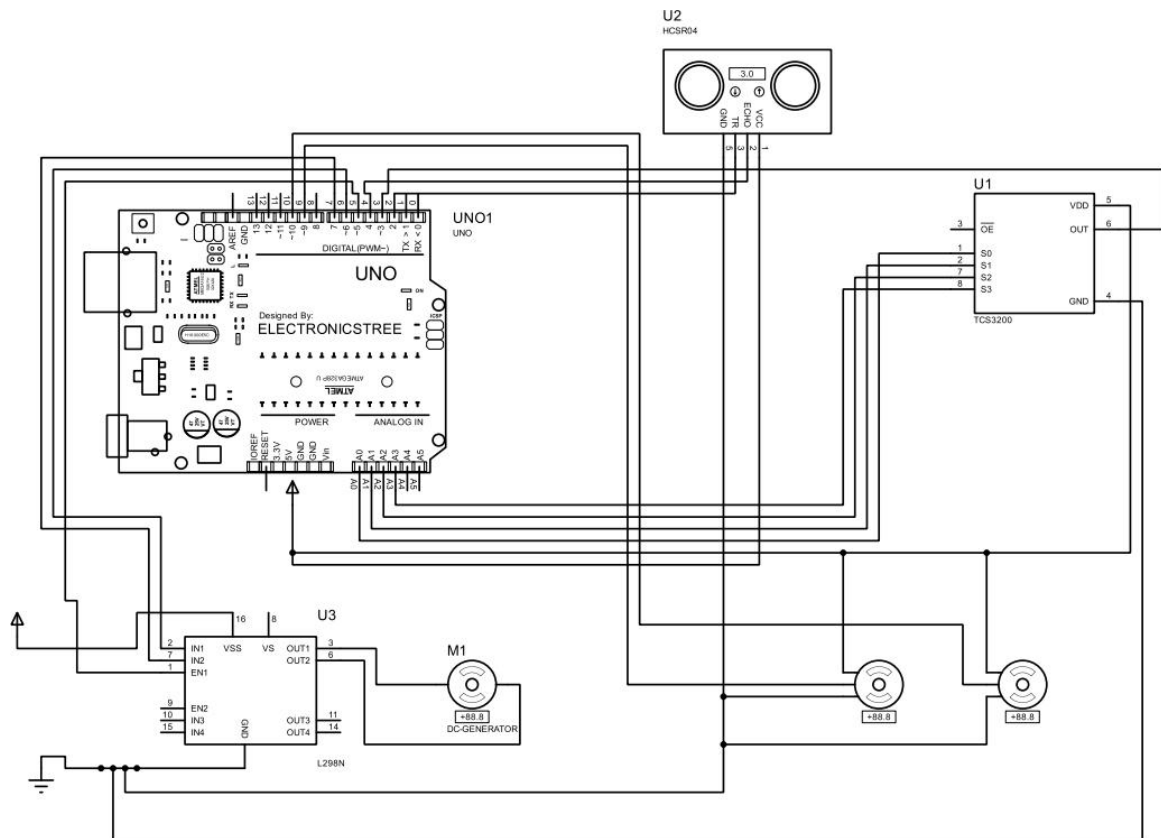


Figure 1: Circuit diagram (made with proteus software)

4.2 Pin mapping

Table 2: Pin map table

Arduino Pin	Connected To
D2	HC-SR04 TRIG
D3	TCS3200 OUT
D4	HC-SR04 ECHO
D5	L298N ENA (PWM)
D6	L298N IN1
D7	L298N IN2
D9	Servo 1 (Left)
D10	Servo 2 (Right)
A0-A3	TCS3200 S0-S3

4.3 Methodology

1. HC-SR04 continuously measures distance. When an object is detected within 12 cm, the belt starts.
2. Belt runs for a timed duration until the object reaches the TCS3200 at 13 cm. Belt stops.
3. TCS3200 reads Red, Green, Blue pulse frequencies. Lower pulse = stronger color.

4. Color is identified (Red, Blue, Yellow, Green) using frequency comparison logic.
5. Based on detected color, the correct servo pre-positions itself.
6. Belt restarts, carries the object to its bin zone. Servo pushes it in.
7. Servos return to home (0°). System resets and waits for next object.

5. Fabrication & Testing Procedure

5.1 Fabrication process

1. Cutting of the wooden board for the base and walls of conveyor.
2. Mounting of motor in a way so that it does not slip out or move.
3. Placement of color sensor and ultrasonic sensor on their appropriate places. TCS3200 mounted mid-belt on a wooden bracket facing downward. HC-SR04 mounted at the entry end of the belt.
4. Cutting and mounting of belt material.
5. Color bins and unknown bin placement.
6. Breadboard and Arduino uno placement on base and wiring according to circuit diagram shown in Figure 1.
7. Wires to Arduino pins according to Table 2.
8. Making of battery using Li-ion cells each of 4.2V in series (voltage increase). Soldering BMS onto it. Common GND wiring between Arduino and L298N.
9. Writing of code and calibration of sensor data. Then we moved onto testing and recalibration.

5.2 Testing process

We simply tested in 2 different lightings with the same objects. Table 3 shows the results obtained from testing.

Table 3: Test results

Test	Object Color	Detected As	Sorted Correctly?	Detected correctly (?/10)	Correct detected percentage
1	Red	Red	Yes	10	100%
2	Blue	Blue	Yes	9	90%
3	Green	Green	Yes	8	80%
4	Yellow	Yellow	Yes	7	70%

- Initially UNKNOWN readings were 100% but then we fixed them by calibration.
- Green box initially misread due to ambient light, it was resolved by adjusting detection logic.
- Slow servo function added to reduce mechanical shock on boxes.

6. Program Code

- Language: Arduino C++
- Libraries used: Servo.h [9]

Table 4: User defined functions for code

Function	Their work
----------	------------

Setup()	Initializes all pins, attaches servos to home position, starts serial monitor
Loop()	Continuously runs the state machine and prints live distance and color to serial monitor
setMotor(int speed)	Starts or stops the conveyor belt by writing direction and PWM speed to L298N
readDistance()	Fires HC-SR04 ultrasonic pulse and returns measured distance in cm
slowServo(Servo &srv, int targetAngle)	Moves a servo smoothly from its current angle to a target angle one degree at a time with a delay between each step
executeRawColorLogic()	Reads R, G, B pulse frequencies from TCS3200 and returns the detected color
printColorLabel(ObjectColor c)	Prints the current color name to the Serial Monitor for debugging

```

#include <Servo.h>
// PIN DEFINITIONS
#define COLOR_S0      A0
#define COLOR_S1      A1
#define COLOR_S2      A2
#define COLOR_S3      A3
#define COLOR_OUT      3
#define TRIG_PIN      2
#define ECHO_PIN       4
#define L298N_ENA      5
#define L298N_IN1      6
#define L298N_IN2      7
#define SERVO_LEFT_PIN 9    // Handles Blue (21-25cm) & Red (25cm)
#define SERVO_RIGHT_PIN 10 // Handles Yellow/Orange (24-25cm) & Green (25cm)
Servo servoLeft;        Servo servoRight;

// SYSTEM SETTINGS & TIMERS
#define SPEED_FULL      255
#define SPEED_STOP      0
#define SERVO_HOME      0
#define ULTRASONIC_TRIP 12.0 // cm distance to trigger the start
// Motor Travel Timers
#define TIME_START_TO_SCAN 1300 // Travel time to 13cm (Color Sensor)
#define SERVO_MOVE_DELAY 800 // Wait for servo to physically move
#define TIME_SCAN_TO_BLUE 1600 // Time to 21-25cm drop zone
#define TIME_SCAN_TO_YELLOW 1700 // Time to 24-25cm drop zone
#define TIME_SCAN_TO_GREEN 1750 // Time to 25cm drop zone
#define TIME_SCAN_TO_RED 1750 // Time to 25cm drop zone

// SERVO SPEED CONTROL
#define SERVO_STEP_DELAY_MS 15 // ms to wait between each 1-degree step
// Moves a servo slowly from its current position to a target angle
void slowServo(Servo &srv, int targetAngle) {
    int current = srv.read(); // read where it currently is
    if (current == targetAngle) return; // already there, nothing to do
    int step = (targetAngle > current) ? 1 : -1; // direction: +1 or -1
    while (current != targetAngle) {
        current += step;
        srv.write(current);
        delay(SERVO_STEP_DELAY_MS);
    }
}

// STATE MACHINE & MEMORY

```

```

enum SystemState
{
    STATE_IDLE,
    STATE_MOVING_TO_SCAN,
    STATE_SCANNING,
    STATE_PRE_DEPLOY,
    STATE_MOVING_TO_DROP,
    STATE_DROP_ACTION
};
enum ObjectColor { COLOR_UNKNOWN, COLOR_BLUE, COLOR_RED, COLOR_YELLOW, COLOR_GREEN };
SystemState currentState = STATE_IDLE;
ObjectColor latchedColor = COLOR_UNKNOWN;

unsigned long actionTimer = 0;
unsigned long targetTravelTime = 0;
unsigned long lastSerialPrint = 0;

void setup()
{
    Serial.begin(9600);
    pinMode(COLOR_S0, OUTPUT); pinMode(COLOR_S1, OUTPUT);
    pinMode(COLOR_S2, OUTPUT); pinMode(COLOR_S3, OUTPUT);
    pinMode(COLOR_OUT, INPUT);
    digitalWrite(COLOR_S0, HIGH);
    digitalWrite(COLOR_S1, LOW); // 20% frequency scaling
    pinMode(TRIG_PIN, OUTPUT); digitalWrite(TRIG_PIN, LOW);
    pinMode(ECHO_PIN, INPUT); pinMode(L298N_ENA, OUTPUT);
    pinMode(L298N_IN1, OUTPUT); pinMode(L298N_IN2, OUTPUT);
    servoLeft.attach(SERVO_LEFT_PIN); servoRight.attach(SERVO_RIGHT_PIN);
    slowServo(servoLeft, SERVO_HOME); slowServo(servoRight, SERVO_HOME);
    delay(500);
    setMotor(SPEED_STOP);
}

void loop() {
    // LIVE SERIAL MONITOR UPDATE (Distance & Memory)
    if (millis() - lastSerialPrint > 500) {
        float liveDist = readDistance();
        Serial.print("Dist: ");
        Serial.print(liveDist, 1);
        Serial.print(" cm | Mem: ");
        printColorLabel(latchedColor);
        lastSerialPrint = millis();
    }

    switch (currentState)
    {
        case STATE_IDLE: {
            float distance = readDistance();
            if (distance > 0.0 && distance <= ULTRASONIC_TRIP)
            {
                Serial.println("\n[SYSTEM] Object Detected! Belt Starting...");
                setMotor(SPEED_FULL);
                actionTimer = millis();
                currentState = STATE_MOVING_TO_SCAN;
            }
            break;
        }
        case STATE_MOVING_TO_SCAN: {
            if (millis() - actionTimer >= TIME_START_TO_SCAN)
            {
                setMotor(SPEED_STOP);
                Serial.println("\n[SYSTEM] Object at 13cm. Belt Stopped for Scan.");
                delay(600);
                currentState = STATE_SCANNING;
            }
            break;
        }
        case STATE_SCANNING: {
            latchedColor = COLOR_UNKNOWN;
            latchedColor = executeRawColorLogic();
        }
    }
}

```

```

    if (latchedColor == COLOR_BLUE)    targetTravelTime = TIME_SCAN_TO_BLUE;
    if (latchedColor == COLOR_YELLOW) targetTravelTime = TIME_SCAN_TO_YELLOW;
    if (latchedColor == COLOR_GREEN)   targetTravelTime = TIME_SCAN_TO_GREEN;
    if (latchedColor == COLOR_RED)     targetTravelTime = TIME_SCAN_TO_RED;
    if (latchedColor == COLOR_UNKNOWN)
    { targetTravelTime = TIME_SCAN_TO_RED;
      latchedColor = COLOR_RED;
    }
    currentState = STATE_PRE_DEPLOY;
    break;
}
case STATE_PRE_DEPLOY: {
    if (latchedColor == COLOR_BLUE)
    { slowServo(servoLeft, 90);
    }
    else if (latchedColor == COLOR_YELLOW)
    { slowServo(servoRight, 90);
    }
    else if (latchedColor == COLOR_GREEN)
    { slowServo(servoRight, 180);
    }
    else if (latchedColor == COLOR_RED)
    { delay(100);
    }
    Serial.println("\n[SYSTEM] Servos Locked. Conveyor Restarting...");
    setMotor(SPEED_FULL);
    actionTimer = millis();
    currentState = STATE_MOVING_TO_DROP;
    break;
}
case STATE_MOVING_TO_DROP: {
    if (millis() - actionTimer >= targetTravelTime)
    { setMotor(SPEED_STOP);
      Serial.println("\n[SYSTEM] Drop Zone Reached. Executing final action.");
      currentState = STATE_DROP_ACTION;
    }
    break;
}
case STATE_DROP_ACTION: {
    if (latchedColor == COLOR_BLUE)
    { slowServo(servoLeft, 0);
    }
    else if (latchedColor == COLOR_RED)
    { slowServo(servoLeft, 180);          delay(1000);
      slowServo(servoLeft, 0);
    }
    else if (latchedColor == COLOR_YELLOW)
    { slowServo(servoRight, 180);        delay(1000);
      slowServo(servoRight, 0);
    }
    else if (latchedColor == COLOR_GREEN)
    { slowServo(servoRight, 0);
    }
    delay(SERVO_MOVE_DELAY);
    latchedColor = COLOR_UNKNOWN;
    targetTravelTime = 0;
    Serial.println("\n[SYSTEM] Cycle Complete. Awaiting Next Object.");
    currentState = STATE_IDLE;
    break;
}
}
}
// HARDWARE DRIVERS
void setMotor(int speed)
{ if (speed > 0) {
    digitalWrite(L298N_IN1, HIGH);    digitalWrite(L298N_IN2, LOW);

```

```

    analogWrite(L298N_ENA, speed);
} else {
    digitalWrite(L298N_IN1, LOW);    digitalWrite(L298N_IN2, LOW);
    analogWrite(L298N_ENA, 0);
}
}

float readDistance() {
    digitalWrite(TRIG_PIN, LOW); delayMicroseconds(4);
    digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long echo = pulseIn(ECHO_PIN, HIGH, 30000UL);
    if (echo == 0) return -1.0;
    return echo * 0.0343 / 2.0;
}

ObjectColor executeRawColorLogic()
{ int R = 0, G = 0, B = 0;
  digitalWrite(COLOR_S2, LOW);    digitalWrite(COLOR_S3, LOW);
  R = pulseIn(COLOR_OUT, LOW); delay(20);
  digitalWrite(COLOR_S2, HIGH);    digitalWrite(COLOR_S3, HIGH);
  G = pulseIn(COLOR_OUT, LOW); delay(20);
  digitalWrite(COLOR_S2, LOW);    digitalWrite(COLOR_S3, HIGH);
  B = pulseIn(COLOR_OUT, LOW); delay(20);
  Serial.println("\n=== RAW FREQUENCY SCAN ===");
  Serial.print("R="); Serial.print(R);
  Serial.print(" G="); Serial.print(G);
  Serial.print(" B="); Serial.println(B);

  if (R > 100 && G > 100 && B > 100) {
      Serial.println(" -> Conveyor Belt (Empty)");
      return COLOR_UNKNOWN;
  }
  else if (R < 45 && G < 45 && B < 45)
  { Serial.println(" -> Conveyor Belt (Empty/White)");
    return COLOR_UNKNOWN;
  }
  else if (R < G && G < B && (G - R) > 15 && (G - R) < 60)
  { Serial.println(" -> DETECTED: ORANGE/YELLOW!");
    return COLOR_YELLOW;
  }
  else if (R < G && R < B)
  { Serial.println(" -> DETECTED: RED!");
    return COLOR_RED;
  }
  else if (G < R && G < B)
  { Serial.println(" -> DETECTED:
    GREEN!"); return COLOR_GREEN;
  }
  else if (B < R && B < G)
  { Serial.println(" -> DETECTED: BLUE!");
    return COLOR_BLUE;
  }
  else {
      Serial.println(" -> DETECTED: UNKNOWN");
      return COLOR_UNKNOWN;
  }
}

void printColorLabel(ObjectColor c)
{ switch(c) {
  case COLOR_RED:    Serial.println("RED");    break;
  case COLOR_GREEN:  Serial.println("GREEN");  break;
  case COLOR_BLUE:   Serial.println("BLUE");   break;
  case COLOR_YELLOW: Serial.println("YELLOW/ORANGE"); break;
  default:           Serial.println("WAITING"); break;
}
}

```

```
}
```

7. Challenges and their solutions

Table 5: Challenges faced during project and their implemented solutions.

Challenge	Solution
Servo rotating at their default full speed caused boxes to virtually fly out of bounds.	Implemented code to slow it down so it moves 1° then delay then moves again.
Color sensor returning UNKNOWN	Adjusted pulse threshold values after calibration.
Green misidentified under room lighting, often being mistaken for red instead.	Rewrote ratio-based detection logic
Ultrasonic sensor burnout, it stopped reading data and started to return 0.0cm right before entering the lab for final evaluation. This caused the object to not be detected and kept moving forward without stopping for color detection. Of course, when that failed to happen the servos did not get triggered to sort into bin.	Diagnosed the fault through serial monitor distance readings during evaluation. This was later fixed by replacing the ultrasonic sensor.
BMS on the battery burned out once, although it should have protected the battery and the circuit. Instead, 2 cells were damaged and had to be replaced. The new BMS had proper voltage but it tend to have really low output in current. Not enough to drive an Arduino as well as a motor driver.	Switched to a 12V DC wall adapter for powering the motor driver, resolving current deficiency, later added the battery back into the circuit.
Missing PWR jumper block on the original motor shield L293D. Using it in that form, by placing it over the Arduino would have instead fried the microprocessor out.	Had to replace it to L298N motor driver.
Color sensor showed different values in different light, which was not an issue for the most part but working in too dark a room actually gave absolutely wrong sorting (because we calibrated the sensor in sufficient light)	Calibrating the color sensor once more. Future fix is to implement dark/ambient compensation. Or enclose the sensor in a veil/box to block external light from messing with the results.
Conveyor belt was initially too stiff for the motor to drive. Fixed the stiffness, but trying to fix the motor shaft together with super glue actually caused the gear motor to stop working. Replacement motor was installed last minute without proper mounting, resulting in instability during evaluation	Replaced the gear motor and resolved belt stiffness; proper motor mounting was identified as a critical improvement for future iterations

Conclusions

This project successfully demonstrated the design and implementation of a low-cost, Arduino Uno-based automated color sorting conveyor belt system sorting in real time. The system integrated a TCS3200 color sensor, HC-SR04 ultrasonic sensor, two SG90 servo motors, a 12V DC gear motor, and an L298N motor driver, all coordinated through a structured state machine implemented in Arduino C++.

The core objective of automating the color sorting process was achieved. Objects placed on the conveyor belt were reliably detected with almost 85% accuracy, color-identified, and sorted into Red, Blue, Yellow, and Green bins through coordinated servo actuation.

The development process involved overcoming several practical hardware and software challenges, including sensor failure under pressure, power supply limitations, and ambient lighting effects on color detection accuracy. Each challenge provided valuable hands-on engineering experience in diagnosis and troubleshooting.

Future improvements to the system could include enclosing the color sensor in a light-controlled shroud to eliminate ambient lighting interference. Future improvements to the system could include enclosing the color sensor in a light-controlled shroud to eliminate ambient lighting interference, expanding color recognition to additional colors beyond the current four and replacing time-based delivery logic with real-time distance tracking for greater accuracy and repeatability.

Overall, the project met its stated objectives and provided a thorough practical application of microcontroller programming, sensor integration, motor control, and mechanical assembly within a single cohesive system.

References

- [1] M. Rahman, H. A. Hasan, Y. A. Emon, A. Hossain, A. R. Rokon, and M. H. Bhuyan, "Advanced Color Sorting Conveyor System Using Arduino and TCS3200 Color Sensor for Precise Color Classification," in *2025 4th International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST)*, 11-12 Jan. 2025 2025, pp. 476-481, doi: 10.1109/ICREST63960.2025.10914353.
- [2] A. Deshmukh, M. Nagane, and V. Awatade, "Design & Development of Automatic Conveyor Controlling System for Sorting of Component on Color basis," *Red*, vol. 630, p. 780, 2016.
- [3] M. A. Alaya, Z. Tóth, and A. Géczy, "Applied Color Sensor Based Solution for Sorting in Food Industry Processing," *Periodica Polytechnica Electrical Engineering and Computer Science*, vol. 63, no. 1, pp. 16-22, 01/01 2019, doi: 10.3311/PPee.13058.
- [4] E. J. Morgan, "HC-SR04 Ultrasonic Sensor datasheet." [Online]. Available: <https://www.alldatasheet.com/datasheet-pdf/view/1132204/ETC2/HCSR04.html>
- [5] "TCS3200, TCS3210 Programmable Color Light-to-Frequency Converter Color Sensor." [Online]. Available: <https://www.mouser.com/catalog/specsheets/TCS3200-E11.pdf>
- [6] "SG90 Micro Servo DataSheet." [Online]. Available: https://www.kjell.com/globalassets/mediaassets/701916_87897_datasheet_en.pdf?ref=4287817A7A
- [7] STMicroelectronics, "Datasheet - L298N - Dual full-bridge driver." [Online]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>
- [8] "User Manual Arduino Uno R3." [Online]. Available: <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>
- [9] M. Margolis, "Device Control Servo.h Library." [Online]. Available: <https://docs.arduino.cc/libraries/servo/>